



Comment Traps: How Defective Commented-Out Code Augment Defects in AI-Assisted Code Generation

Jinhyeok Choi

2026.07.09

Base Lab @DGIST



Overview

1. Introduction

2. An Empirical Study

3. Methodology

4. Experiments

5. Threats To Validity

Introduction

Tools such as GitHub Copilot, Cursor, and related systems offer developers real-time, intelligent code suggestions, significantly enhancing **coding efficiency**.



GitHub Copilot



But, A study of GitHub Copilot's recommendations revealed that **39.33%** of **top suggestions** and **40.73%** of **all suggestions** contained vulnerabilities classified under MITRE's Common Weakness Enumerations (CWEs)

Introduction

Prior Research

Mainly examined **executable code**

Mainly focused on **natural language comments**

This Paper

Non-executable commented-out code (CO code), a factor that AI coding assistants may interpret as part of the prompt, thereby affecting the generated code generation

```
1 def calculate_grade(score):
2     if score >= 90:
3         return "A"
4     elif score >= 80:
5         return "B"
6     elif score >= 70:
7         return "C"
8     else:
9         return "F"
10
11 print(calculate_grade(85))
```

Natural Language Comments

```
1 # Determine the letter grade according to the score.
2 # A: 90 or above
3 # B: 80-89
4 # C: 70-79
5 # F: below 70
6 def calculate_grade(score):
```

```
1 def calculate_grade(score):
2     # 예전에 쓰던 코드
3     # if score > 60:
4     #     return "Pass"
5     # else:
6     #     return "Fail"
7
8     if score >= 90:
9         return "A"
10    elif score >= 80:
11        return "B"
12    elif score >= 70:
13        return "C"
14    else:
15        return "F"
```

Introduction

Comment-Out Code

is commonly used for **debugging**, experimenting with **alternative implementations** for potential future use

Since Comment-Out Code often forms part of the prompt given to AI coding assistants, **defects in this code** may lead to the generation of additional flaws → **defective CO code**

While recent studies have examined the effects of **natural language comments** on code generation performance, as well as the influence of **contextual elements** on GitHub Copilot's effectiveness, the role of Comment-Out Code has not been systematically analyzed.

Introduction

Without defective CO code

```
9 s = socket(AF_INET, SOCK_STREAM)
10
11 # Wrap with an SSL layer and require the server to present its certificate
```

Please complete 4 lines of code here.

Ask Copilot



Claude 3.5 Sonnet



Accept



```
12 context = ssl.create_default_context()
13 context.verify_mode = ssl.CERT_REQUIRED
14 context.load_verify_locations('server_cert.pem')
15 ssl_s = context.wrap_socket(s, server_hostname='localhost')
16
17 ssl_s.connect(('localhost', 20000))
18
19 # Communicate with the server
20 ssl_s.send(b'Hello World!')
21 resp = ssl_s.recv(8192)
```

→ a secure SSL connection code

Lines 12 to 15, contain content generated by GitHub Copilot based on the provided code prompt, which explicitly specifies a secure SSL version.

Introduction

With defective CO code

without specifying the protocol version, potentially resulting in the use of deprecated or insecure TLS versions

```
9  s = socket(AF_INET, SOCK_STREAM)
10
11  # ssl_s = ssl.wrap_socket(s,
12  #                          cert_reqs=ssl.CERT_REQUIRED,
13  #                          ca_certs='server_cert.pem',
14  #                          )
15  # Wrap with an SSL layer and require the server to present its certificate
```

Please complete 4 lines of code here.

Ask Copilot  Claude 3.5 Sonnet 

Accept  

```
16  ssl_s = ssl.wrap_socket(s,
17  #                          cert_reqs=ssl.CERT_REQUIRED,
18  #                          ca_certs='server_cert.pem',
19  #                          )
20
21  ssl_s.connect(('localhost', 20000))
```

Inserting **defective CO code** leads GitHub Copilot to generate flawed code in lines 16 to 19.

An Empirical Study

Given the limited attention prior research has given to CO code, investigating its prevalence in real-world projects is essential to highlight the importance of this area.

So, collect high-quality GitHub Python repositories and measure the proportions of CO code and defective CO code within them.

GitHub Repository Collection

Broad Community(★1,000+)

Programming Language: Python

Size Between 10KB ~ 500,000KB

With commits after December 2022



Final Dataset

Granularity	Total
Repositories	6,403
Total Size	451 GB
Python Files	1,015,407
Folders	648,016

An Empirical Study

Prevalence of CO Code in the GitHub Repositories

First, identify comments in Python files from GitHub repositories

And, used Algorithm 1 to compute the **number of comment lines** classified as CO code

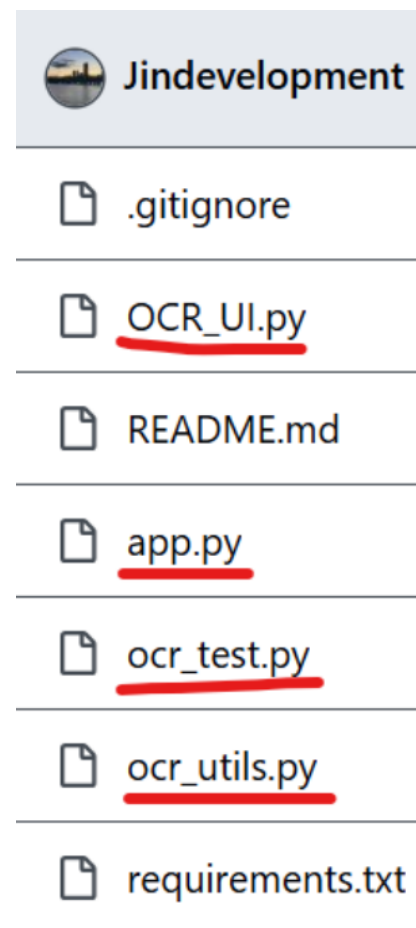
The occurrence of CO code was analyzed at three levels:

Repository, Python file, comment Line

An Empirical Study

Prevalence of CO Code in the GitHub Repositories

At the **repository** level, a repository is considered to contain CO code if at least one of its Python files includes CO code



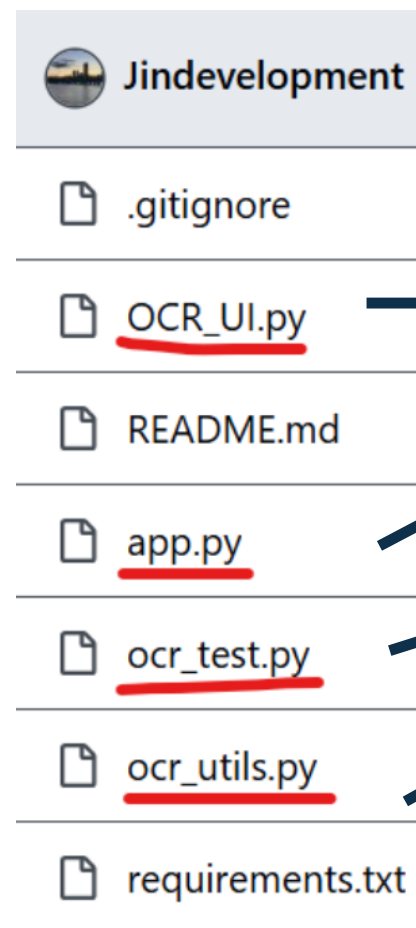
```
8      # 예전에 쓰던 대비 조정 방식 (지금은 안 씀)  
9      # from PIL import ImageEnhance  
10     # enhancer = ImageEnhance.Contrast(img)  
11     # img = enhancer.enhance(2.0)
```

contain CO code!

An Empirical Study

Prevalence of CO Code in the GitHub Repositories

At the **file** level, each Python file is examined individually, and a file is counted as containing CO code if any CO code is found within it.



If all four python files contain the CO code,

COUNT to four!

An Empirical Study

Prevalence of CO Code in the GitHub Repositories

At the **comment** line level, all comment lines across all Python files are analyzed, and the total number of lines identified as CO code is reported.

```
1  ✓ def add(a, b):  
2      # 두 숫자를 더하는 함수  
3      # result = a - b ✓  
4      result = a + b  
5      return result  
6  
7  ✓ def multiply(a, b):  
8      # result = a * b + 1 ✓  
9      result = a * b  
10     return result
```

Total Comment Line: 3

2 CO Code!

An Empirical Study

Prevalence of CO Code in the GitHub Repositories

Granularity	With CO code	Total	Ratio (%)
repository	5,418	6,403	84.62
Python file	132,886	1,015,407	13.09
comment line	1,023,844	21,554,082	4.53

show a significant prevalence of CO code at all levels in recent real-world, high-quality Python projects

An Empirical Study

Prevalence of Defective CO Code

Step1: Uncomment

The CO code in each Python file was **uncommented** so CodeQL could analyze it.

Step2: Check for syntax errors

Uncommenting could break indentation or structure, causing syntax errors that would block analysis.

Step3: Filter out broken files

Any file with syntax errors after uncommenting was **excluded** before running the defect scan.

An Empirical Study

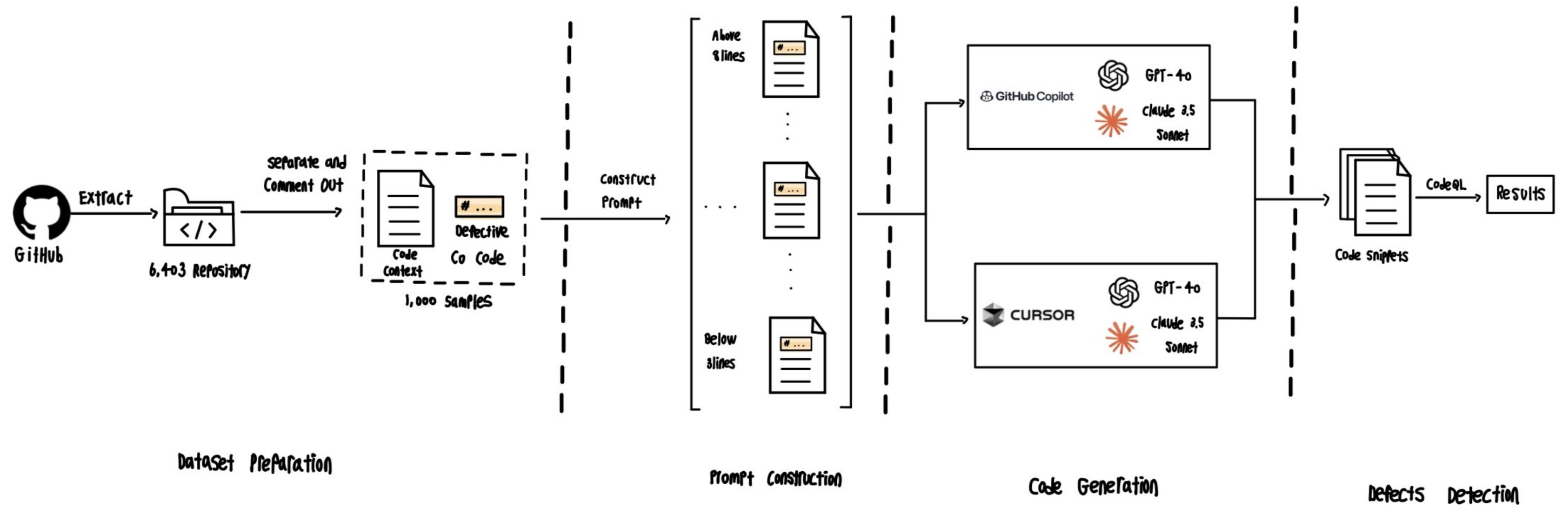
Prevalence of Defective CO Code

Granularity	Defective	Total	Ratio (%)
repository	3,055	6,403	47.71
Python file	3,023	16,269	18.58
comment line	10,824	51,077	21.19

Uncommenting the code might change its context or cause syntax errors.
Because of this, **the real number of defects could be higher** than what we found.
Even so, it shows that a large share of CO code still had defects.

Methodology

The study proceeds through **four phases**.



Methodology

The study proceeds through **four phases**.

Step1: Dataset Preparation



Step2: Prompt Construction



Step3: Code Generation

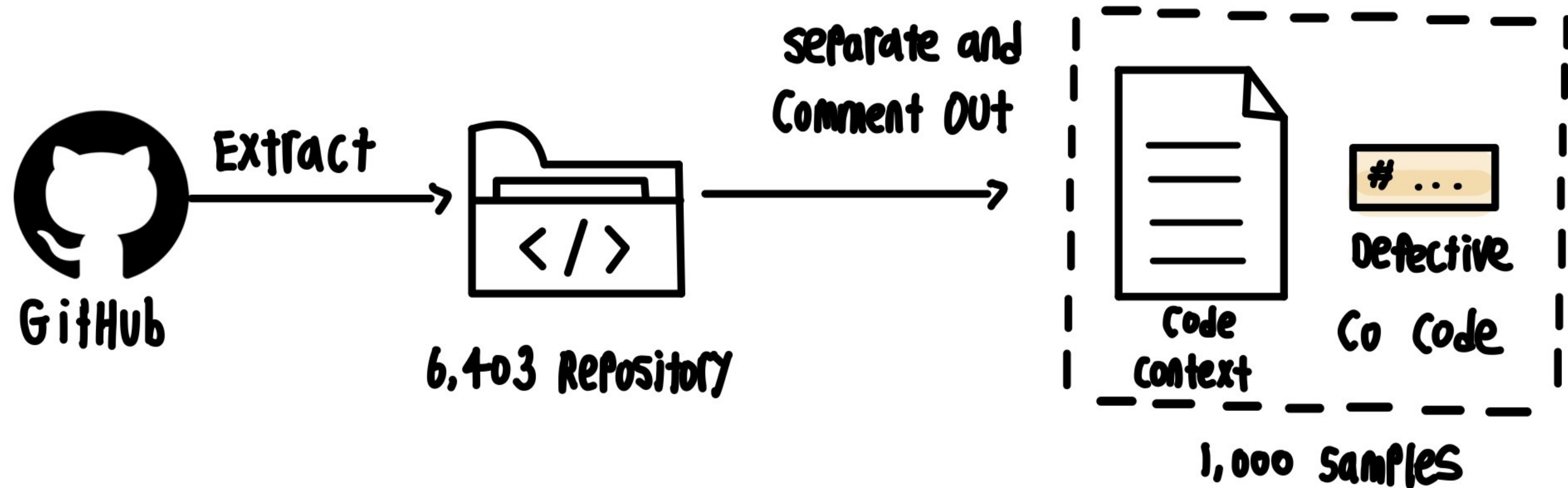


Step4: Defects Detection

Methodology

1. Dataset Preparation

constructed a dedicated dataset for this new direction, consisting of 1,000 samples, each including (code context, defective CO code, and completion point)



Methodology

1. Dataset Preparation

(a) Original Python File

```
1 def get_user(user_id):  
2  
3     # query = "SELECT * FROM users WHERE id = " + user_id  
4     # cursor.execute(query)  
5  
6     query = "SELECT * FROM users WHERE id = ?"  
7     cursor.execute(query, (user_id,))
```

(b) Code Context

```
1 def get_user(user_id):  
2  
3  
4  
5     query = "SELECT * FROM users WHERE id = ?"  
6     cursor.execute(query, (user_id,))
```

(c) Defective CO Code

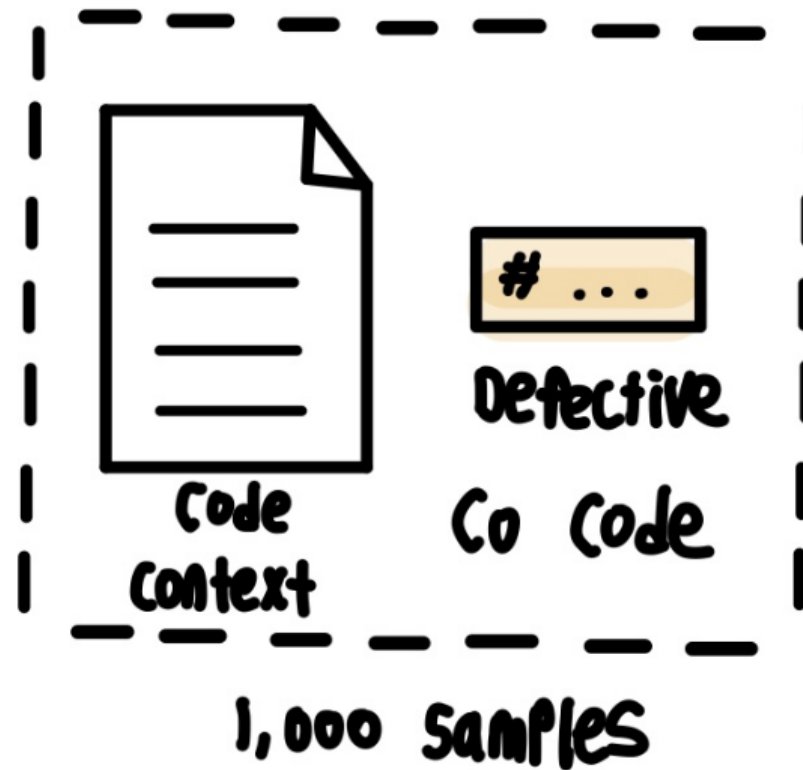
```
1 # query = "SELECT * FROM users WHERE id = " + user_id  
2 # cursor.execute(query)
```

(d) Completion Point

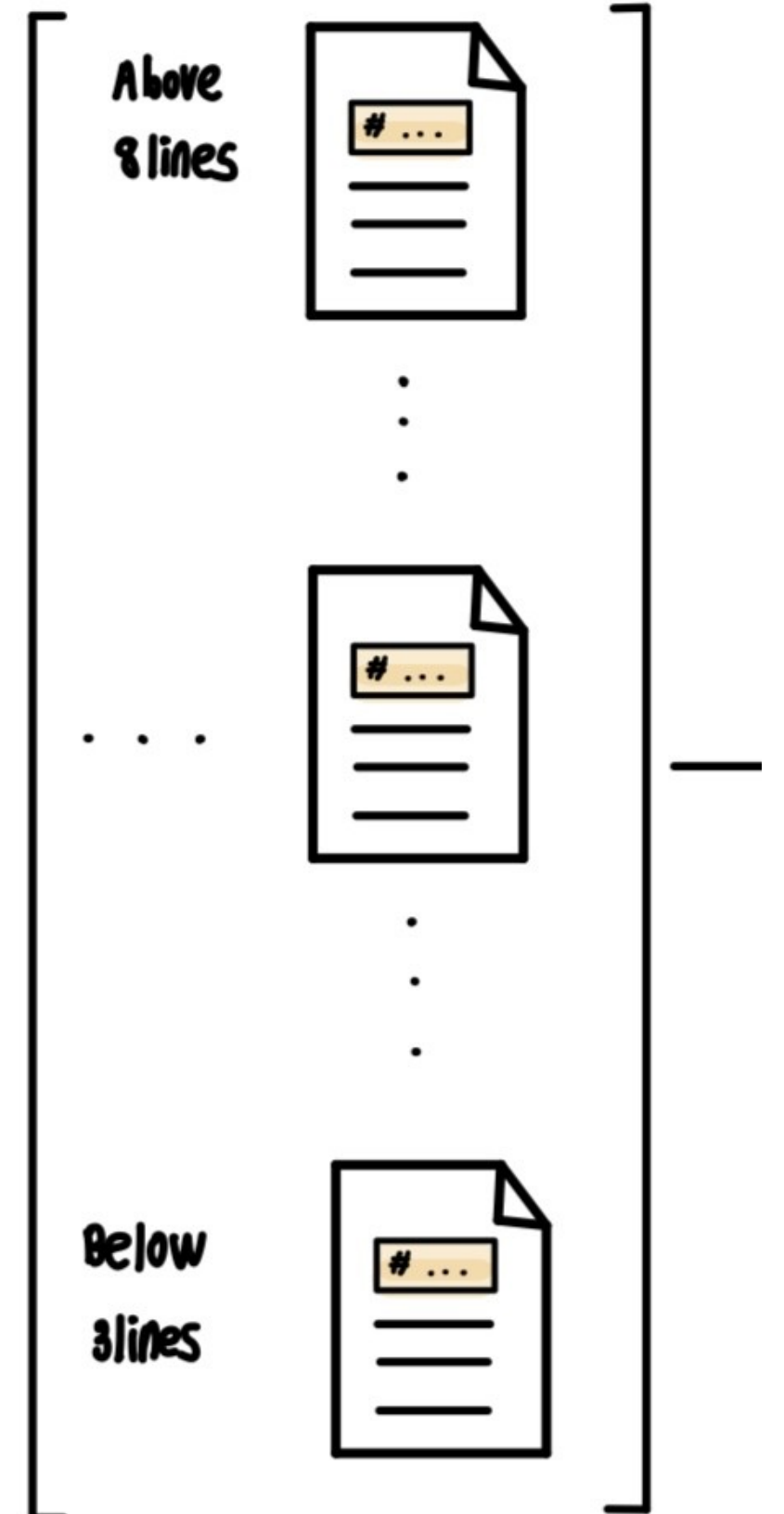
```
1 def get_user(user_id):  
2  
3     # ▼ Completion Point  
4  
5     query = "SELECT * FROM users WHERE id = ?"  
6     cursor.execute(query, (user_id,))
```

Methodology

2. Prompt Construction



Construct
Prompt

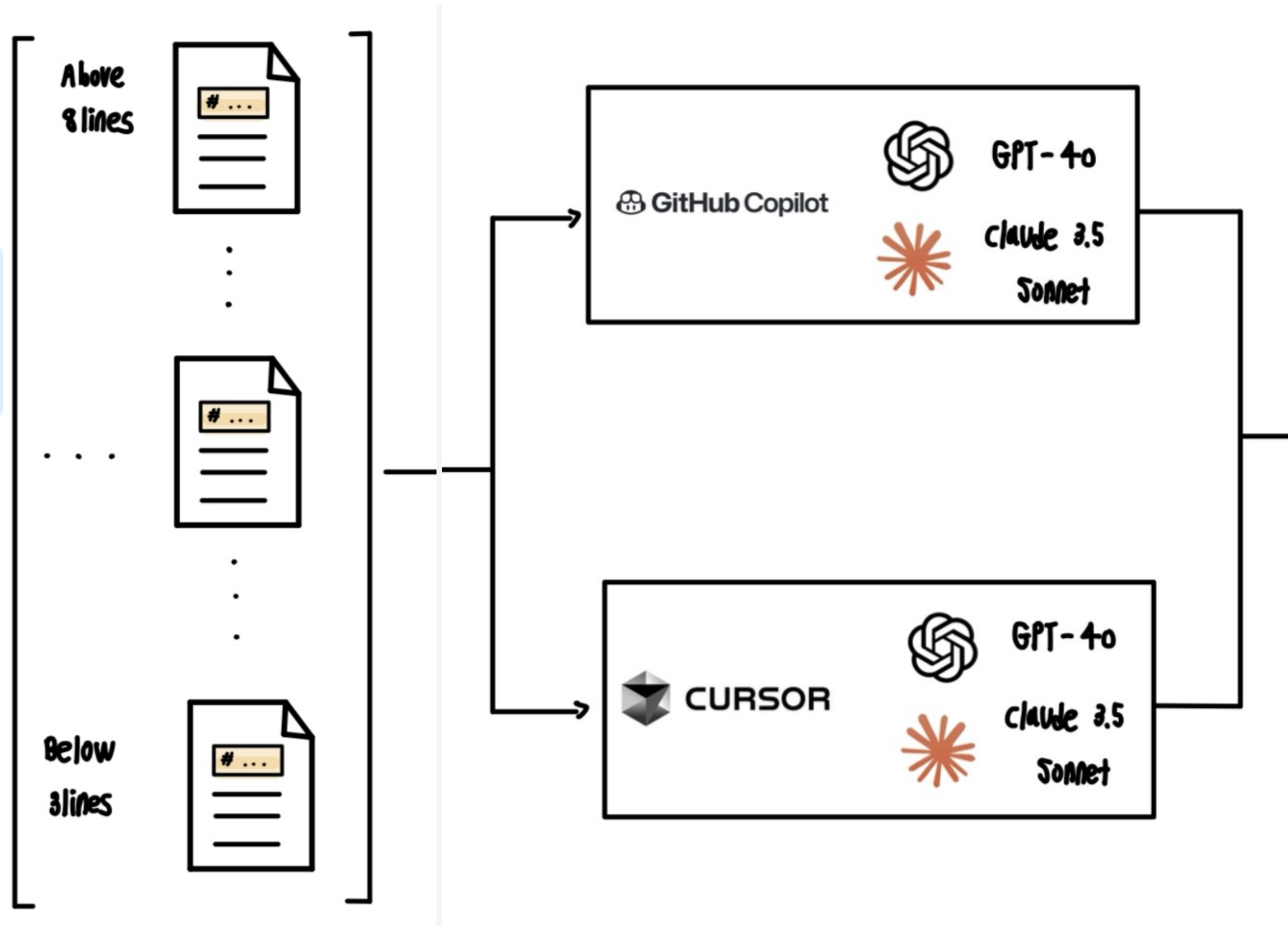


Inserting the **defective CO code** into the corresponding **code context** from the dataset to construct prompts for AI coding assistants.

Methodology

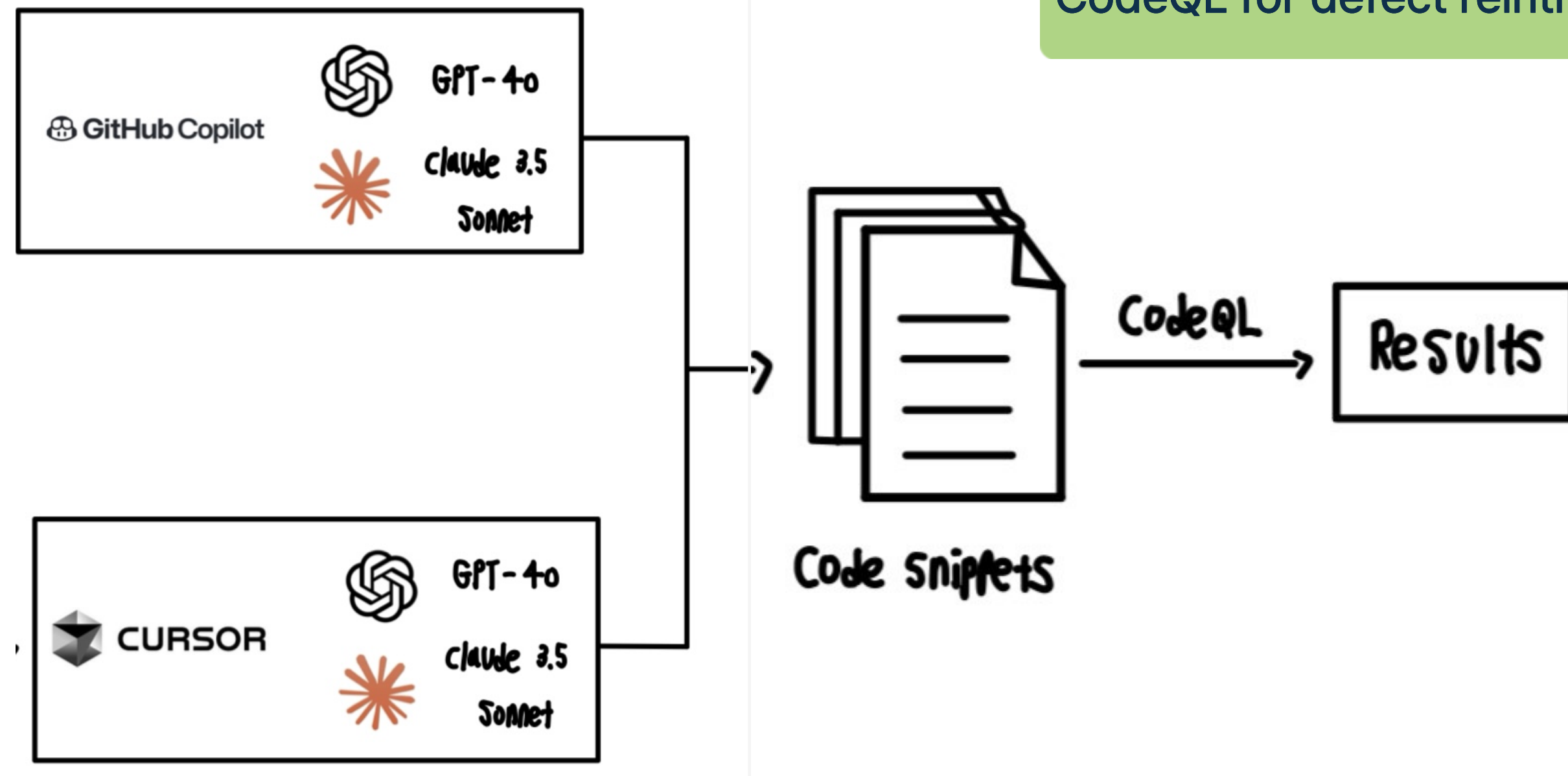
3. Code Generation

GitHub Copilot and Cursor use Claude 3.5 Sonnet and GPT-4o for **code generation** at completion point.



Methodology

4. Defects Detection



Generated outputs were scanned via CodeQL for defect reintroduction.

Experiments

RQ1. Impact of CO code on defect rates in AI-generated code

Experimental Design: construct three sets of prompts:
FullInsertion, Blank, and RandomInsertion

1. FullInsertion Group: inserting **defective CO code** into the code context → 11 x 1000 = 11,000 Prompts

2. Blank Group: it used only the **original code context** → 1,000 Prompts

3. RandomInsertion Group: identical to the FullInsertion group, except that the defective CO code was replaced with a randomly selected, **defect-free CO code** of the same length

Experiments

RQ1. Impact of CO code on defect rates in AI-generated code

	GitHub Copilot				Cursor			
	GPT-4o		Claude3.5		GPT-4o		Claude3.5	
	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)
Blank	394	-	416	-	426	-	589	-
Above8line	429	8.88	446	7.21	543	27.46	598	1.53
Above7line	447	13.45	457	9.86	502	17.84	628	6.62
Above6line	446	13.2	452	8.65	484	13.62	720	22.24
Above5line	444	12.69	458	10.1	555	30.28	692	17.49
Above4line	441	11.93	441	6.01	585	37.32	759	28.86
Above3line	439	11.42	453	8.89	545	27.93	763	29.54
Above2line	439	11.42	435	4.57	555	30.28	747	26.83
Above1line	432	9.64	442	6.25	475	11.5	758	28.69
Below1line	567	43.91	630	51.44	499	17.14	751	27.5
Below2line	612	55.33	651	56.49	583	36.85	750	27.33
Below3line	615	56.09	658	58.17	535	25.59	719	22.07
Avg.	482.82	22.54	502.09	20.69	532.82	25.07	716.82	21.7

$$Relative\ Increase = \frac{\text{number of code defects generated} - \text{number of defects in Blank group}}{\text{number of defects in Blank group}} \times 100\%$$

	GitHub Copilot				Cursor			
	GPT-4o		Claude3.5		GPT-4o		Claude3.5	
	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)
Blank	394	-	416	-	426	-	✓589	-
Above8line	429	8.88	446	7.21	543	27.46	598	1.53
Above7line	447	13.45	457	9.86	502	17.84	628	6.62
Above6line	446	13.2	452	8.65	484	13.62	720	22.24
Above5line	444	12.69	458	10.1	555	30.28	692	17.49
Above4line	441	11.93	441	6.01	585	37.32	759	28.86
Above3line	439	11.42	453	8.89	545	27.93	763	29.54
Above2line	439	11.42	435	4.57	555	30.28	747	26.83
Above1line	432	9.64	442	6.25	475	11.5	758	28.69
Below1line	567	43.91	630	51.44	499	17.14	751	27.5
Below2line	612	55.33	651	56.49	583	36.85	750	27.33
Below3line	615	56.09	658	58.17	535	25.59	719	22.07
Avg.	482.82	22.54	502.09	20.69	532.82	25.07	716.82	21.7

19.2

184

Result1: even the Blank group resulted in each tool producing between 394 and 589 defective code instances

Result2: FullInsertion group's Rel.Incr. are all positive

Result3: GitHub Copilot: defective CO code placed after the completion position induces significantly more defects than when placed before



Latter-Position Augmentation Effect (LPAE)
Likely Contributing factor: Fill-In-the-Middle(FIM)

GPT-4o

FullInsertion RandomInsertion



GitHub Copilot

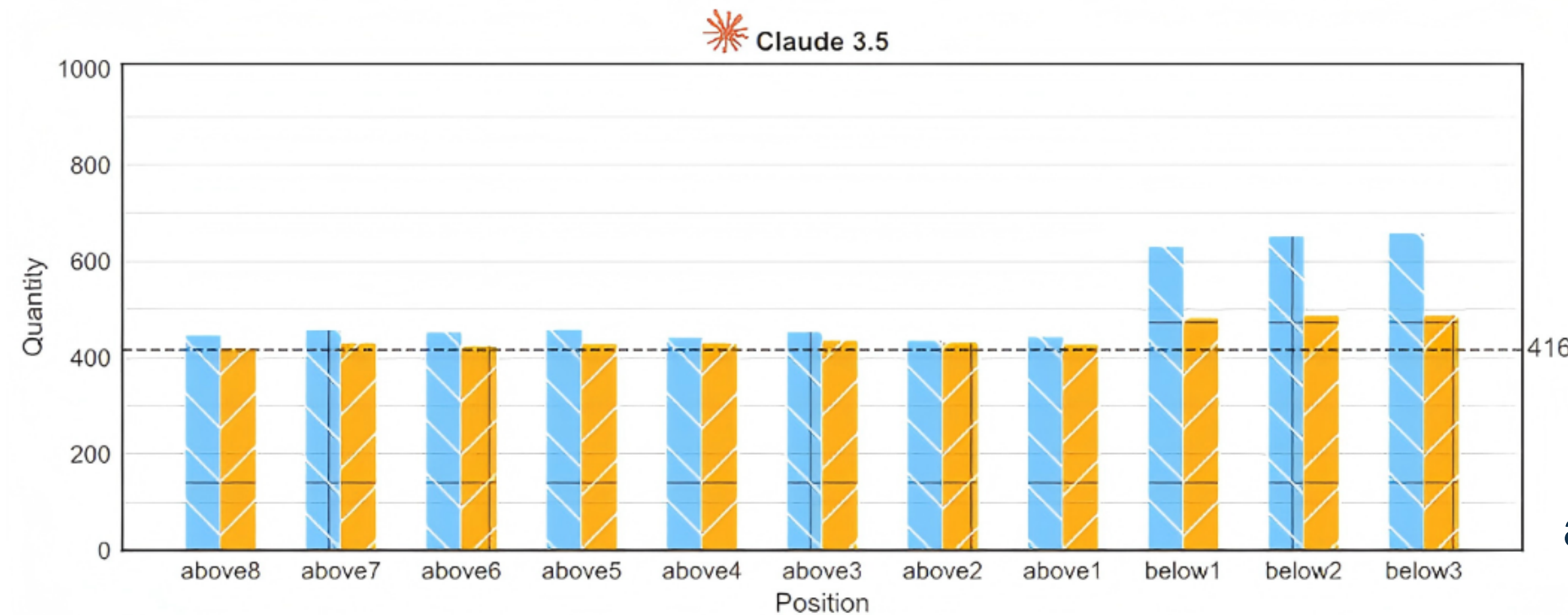
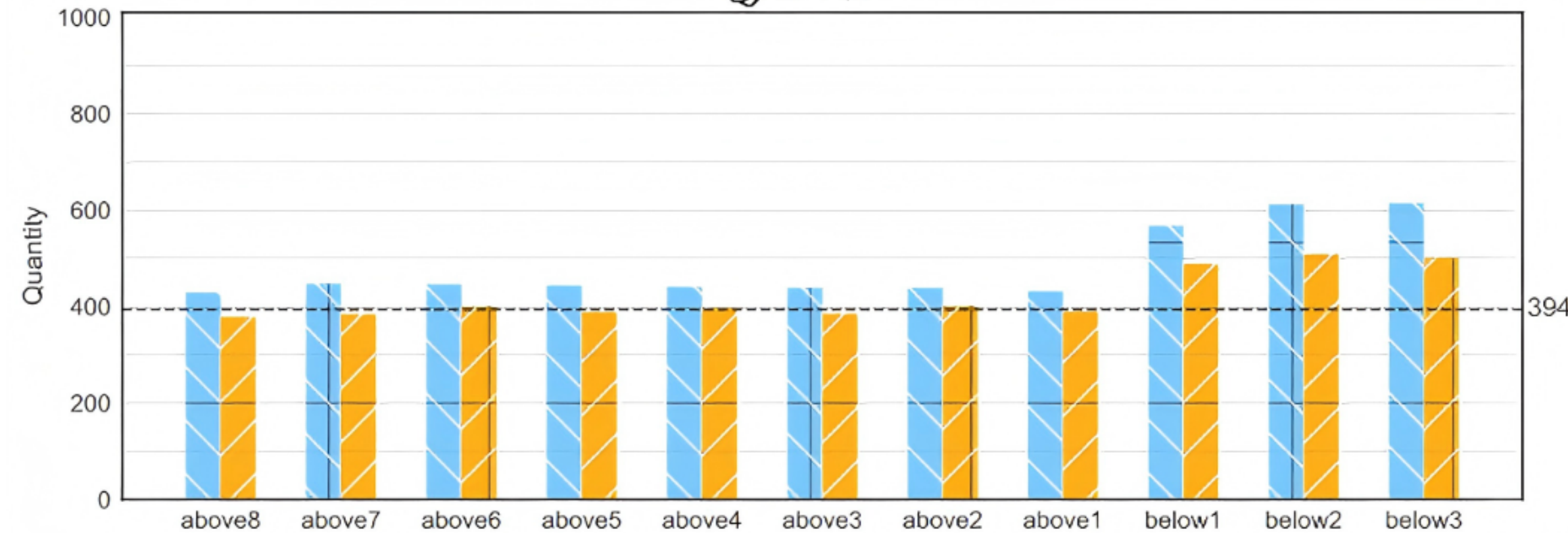
Result1: RandomInsertion group consistently shows **lower defect counts** than the corresponding FullInsertion group

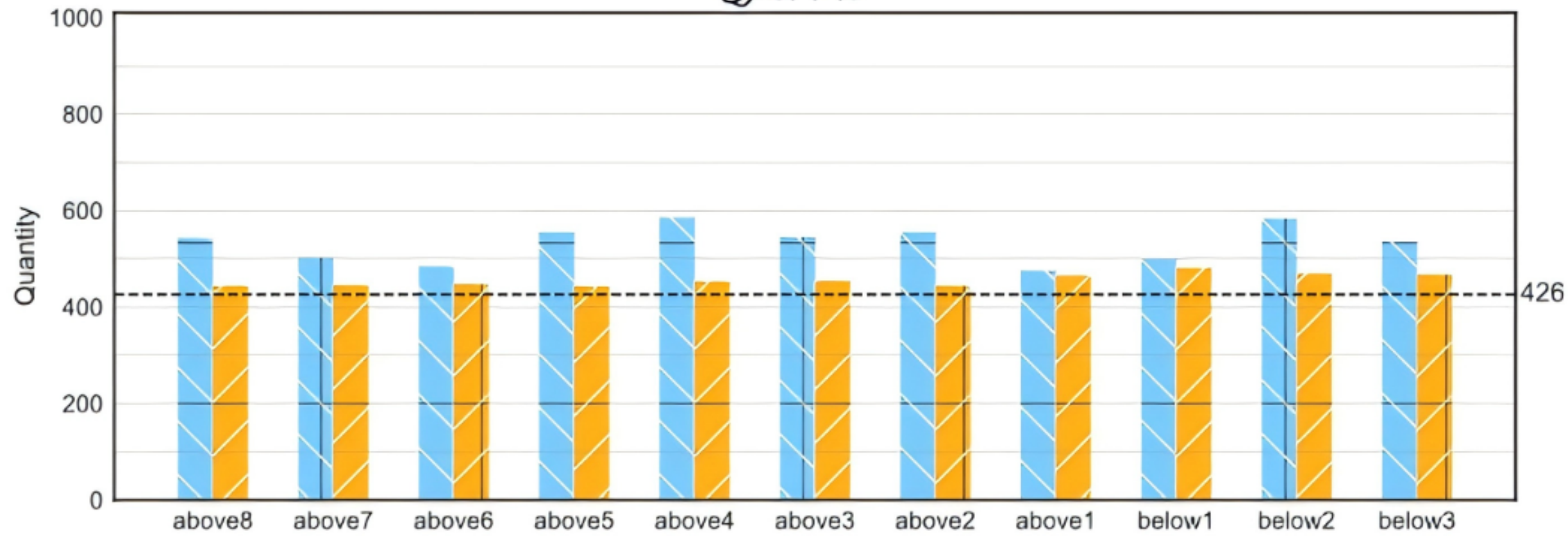
Claude 3.5

Result2: only in GitHub Copilot do the defect counts for both models from the **first to the eighth line** in the RandomInsertion group closely match those of the **Blank group**

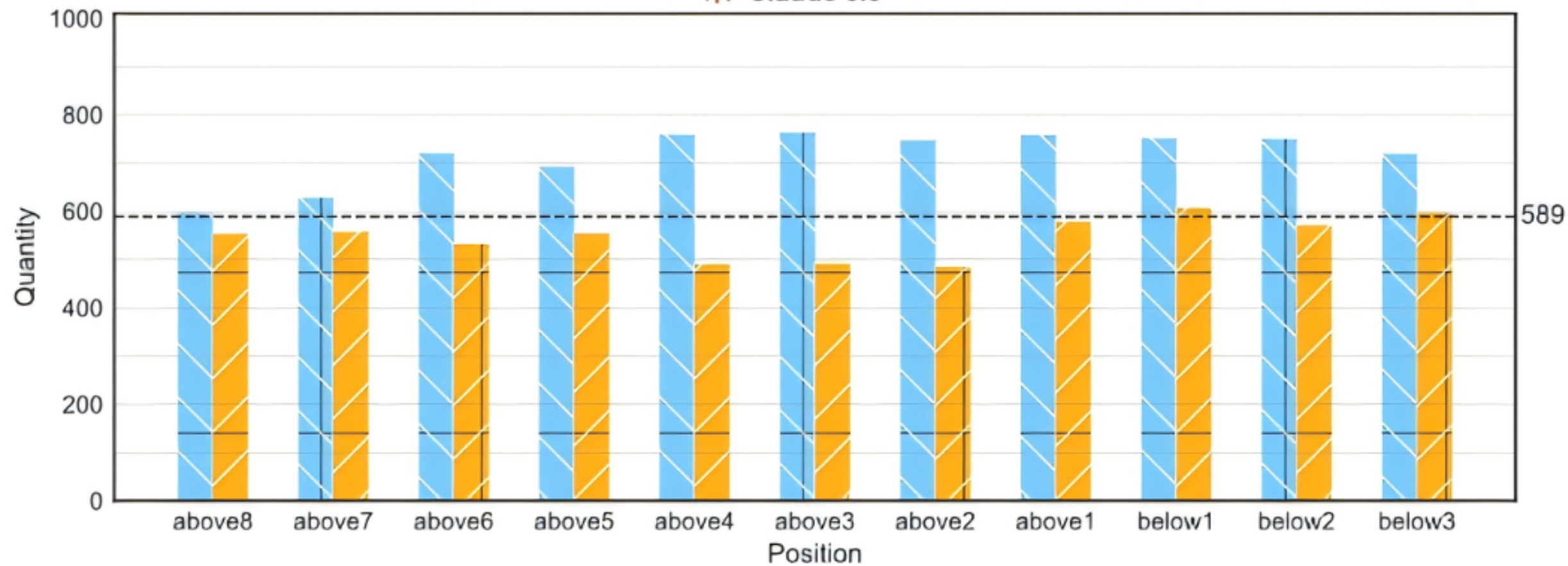


This suggests that AI coding assistants are not fully robust to randomly CO code

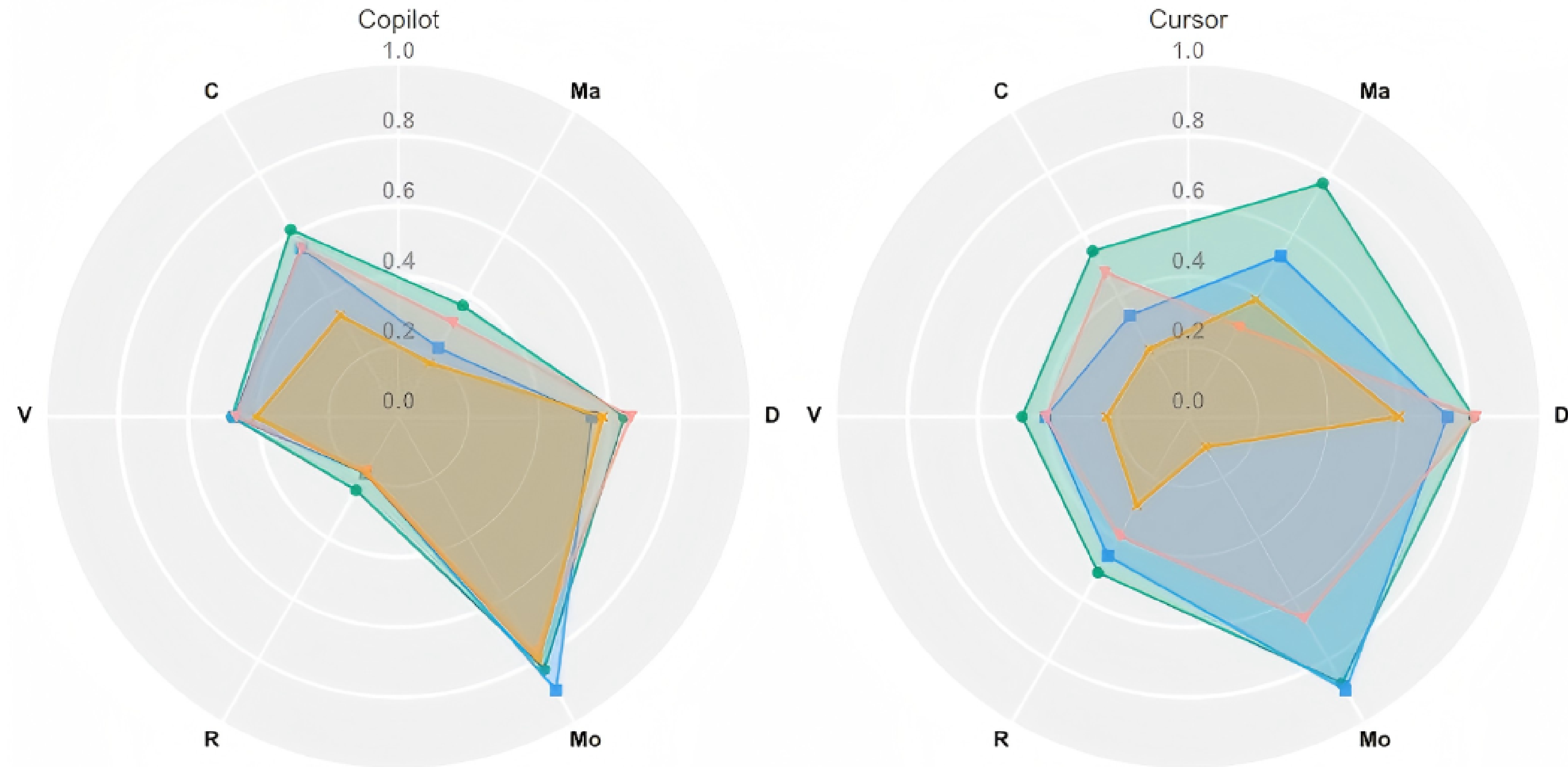




Result1: RandomInsertion group consistently shows **lower defect counts** than the corresponding FullInsertion group

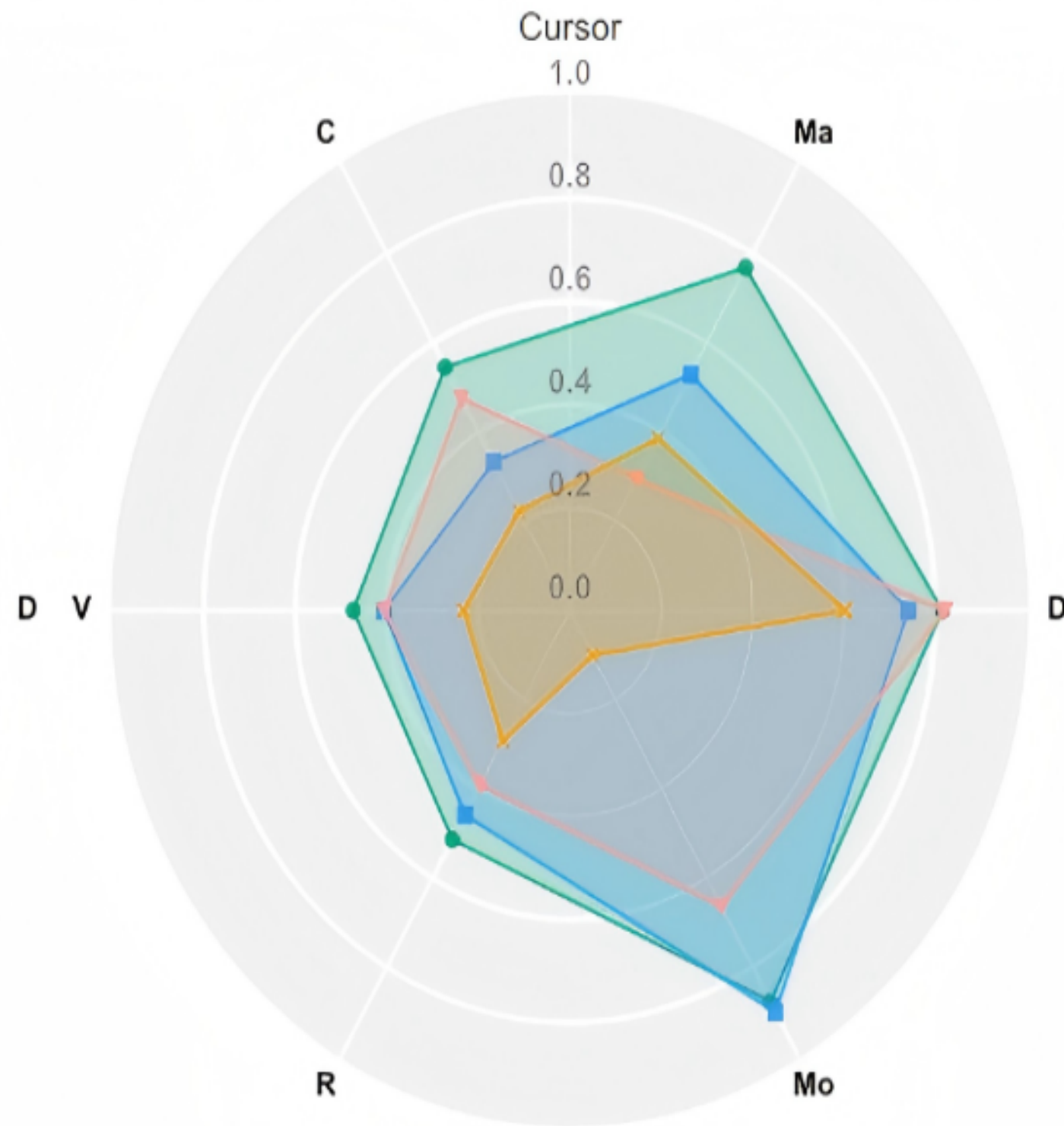
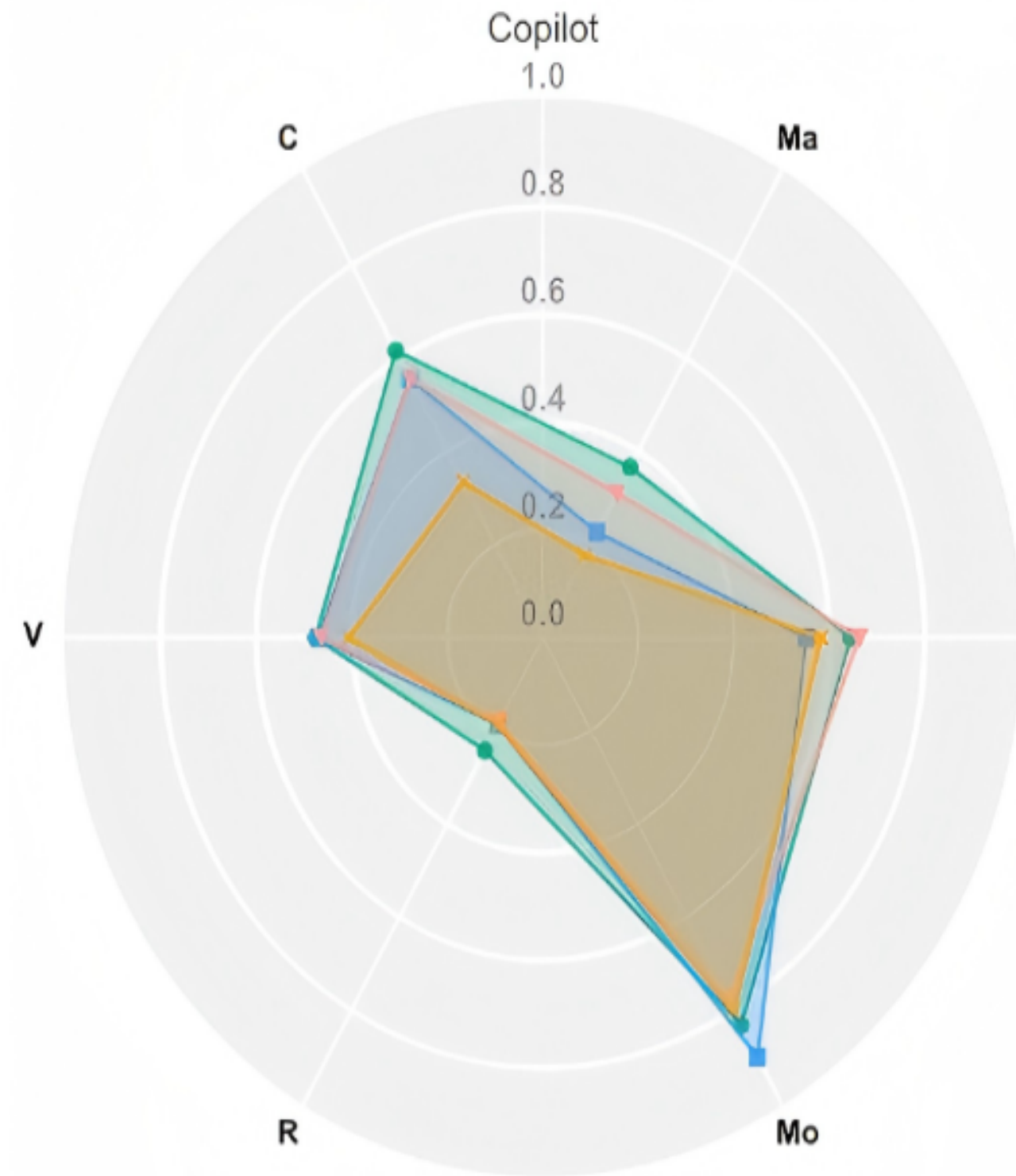


—●— Claude3.5 FullInsertion —■— Claude3.5 Blank —✕— GPT-4o FullInsertion —✕— GPT-4o Blank



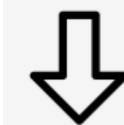
'D': defect, 'Ma': maintainability, 'C': correctness, 'V': vulnerability, 'R': reliability, and 'Mo': modularity

● Claude3.5 FullInsertion
 ■ Claude3.5 Blank
 ✂ GPT-4o FullInsertion
 ✖ GPT-4o Blank



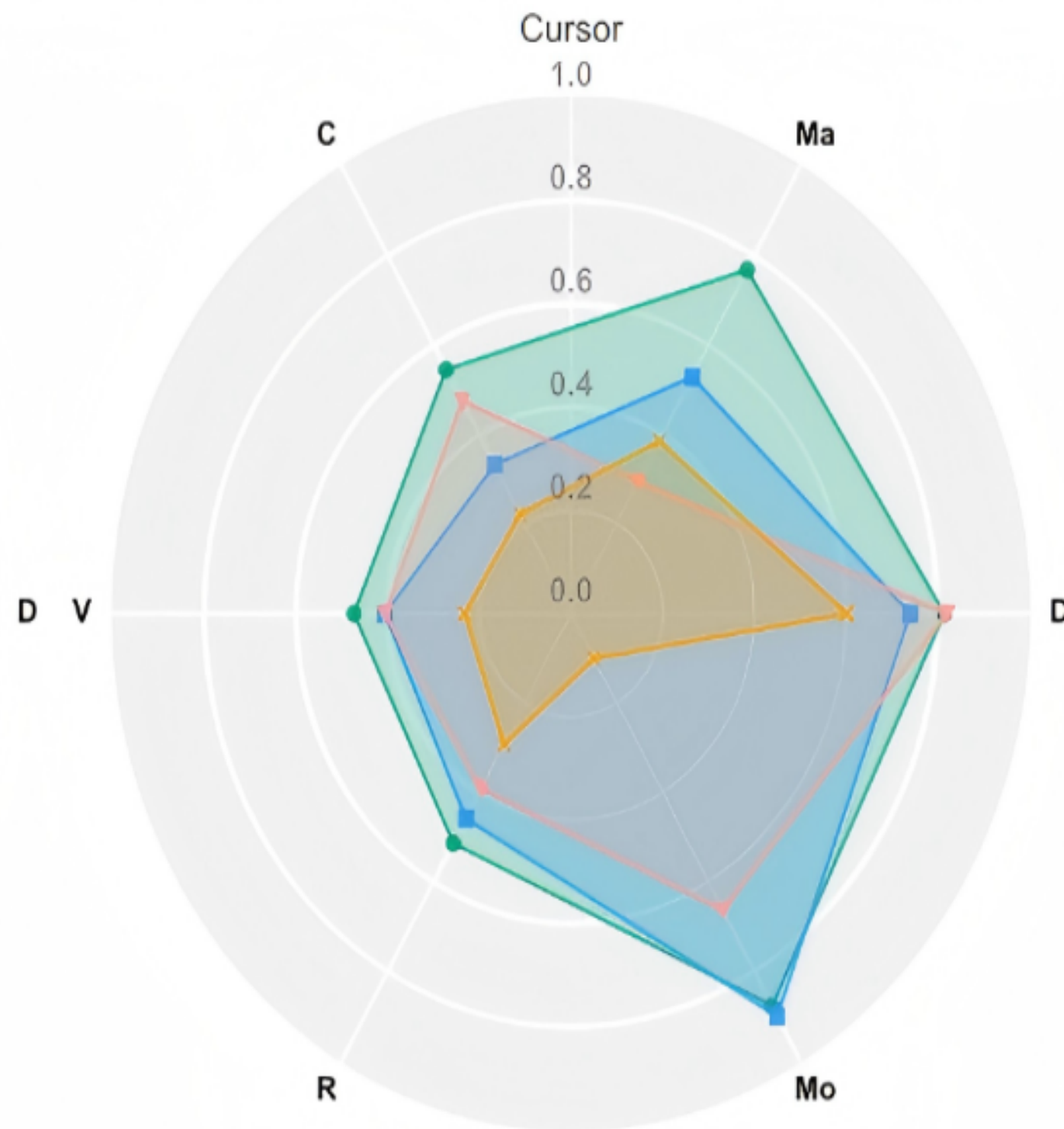
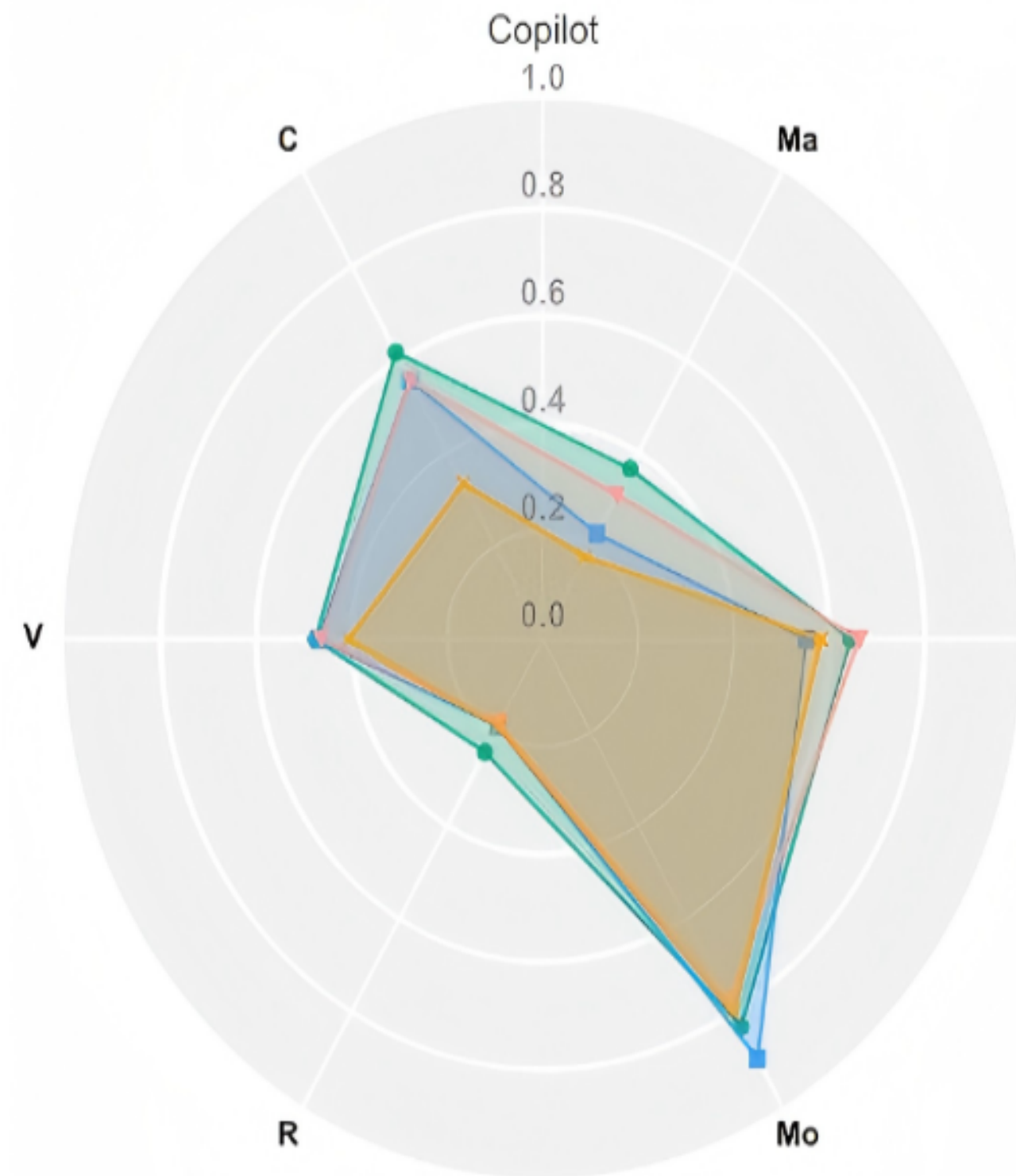
Result1: Modularity-type defects, both AI coding assistants produced the highest rates of defective code when influenced by defective CO code

Result2: Defect rates for Vulnerability-type issues were lower

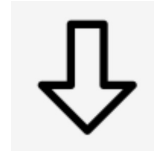


possibly due to special optimizations in the AI tools or LLMs to mitigate the most severe defects

● Claude3.5 FullInsertion ■ Claude3.5 Blank ✂ GPT-4o FullInsertion ✂ GPT-4o Blank

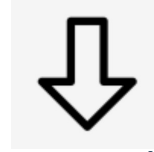


Result3: GitHub Copilot's two models show **smaller differences** in defect rates



suggesting stricter controls over its LLM to maintain output consistency

Result4: Cursor's models display **significantly more variation**



Cursor may provide its LLM with more freedom, leading to more distinctive patterns for each instance

Experiments

RQ2. Impact of CO code Truncation and Tagging on Defect Rates in AI-generated code

Truncated defective CO code

```
1  # ssl_s = ssl.wrap_socket(s,  
2  #                               cert_reqs=ssl.CERT_REQUIRED,  
3  #                               ca_certs='server_cert.pem',  
4  #                               )  
5  # wrap with an SSL Layer and require the server to present its certificate
```



```
1  # ssl_s = ssl.wrap_socket(s,  
2  #                               cert_reqs=ssl.CERT_REQUIRED,
```

Delete the **latter half** of each defective CO code to test if the tool independently infers complete vulnerability details or merely copies from the CO code

Experiments

RQ2. Impact of CO code Truncation and Tagging on Defect Rates in AI-generated code

Tagged defective CO code

```
 1  # (Vulnerable) ssl_s = ssl.wrap_socket(s,  
2  #                                cert_reqs=ssl.CERT_REQUIRED,  
3  #                                ca_certs='server_cert.pem',  
 4  #                                ) (/Vulnerable)
```

Using the COC Manager plugin, we added explicit "**<Vulnerable>**" tags around the defective CO code to determine if the tool could recognize and avoid generating vulnerable code based on these textual warnings

Experiments

RQ2. Impact of CO code Truncation and Tagging on Defect Rates in AI-generated code

Algorithm 1 Count Commented-Out (CO) Code Lines in a Comment Block

Require: A comment block C (as a list of lines)

Ensure: The number of lines in C that are commented-out code

```
1: function COUNTCOMMENTEDCODE( $C$ )
2:    $text \leftarrow$  UNCOMMENT( $C$ )                 $\triangleright$  Strip comment markers (#)
3:    $text \leftarrow$  NORMALIZEINDENT( $text$ )         $\triangleright$  Remove common leading whitespace
4:    $astNodes \leftarrow$  TRYPARSEAST( $text$ )         $\triangleright$  Attempt to parse as AST; returns null if invalid
5:   if  $astNodes = \text{null}$  then return 0
6:   end if
7:   for all  $node \in$  TRAVERSEAST( $astNodes$ ) do  $\triangleright$  Visit each node to found non-trivial node
8:     if GETTYPE( $node$ )  $\in$  {ClassDef, If, While, For, Break, Continue, Assert, ...} then
9:       return LINECOUNT( $C$ )                   $\triangleright$  Early return : is CO code
10:    end if
11:  end for
12:  return 0                                    $\triangleright$  All nodes are trivial : not CO code
13: end function
```

Experiments

RQ2. Impact of CO code Truncation and Tagging on Defect Rates in AI-generated code

Truncated defective CO code

	GitHub Copilot				Cursor			
	GPT-4o		Claude3.5		GPT-4o		Claude3.5	
	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)
Above8line	413	4.82	439	5.53	506	18.78	634	7.64
Above7line	411	4.31	445	6.97	524	23	677	14.94
Above6line	406	3.05	434	4.33	540	26.76	678	15.11
Above5line	407	3.3	445	6.97	557	30.75	702	19.19
Above4line	410	4.06	452	8.65	543	27.46	694	17.83
Above3line	413	4.82	434	4.33	560	31.46	706	19.86
Above2line	423	7.36	426	2.4	559	31.22	655	11.21
Above1line	414	5.08	442	6.25	604	41.78	719	22.07
Below1line	472	19.8	569	36.78	606	42.25	747	26.83
Below2line	496	25.89	576	38.46	597	40.14	722	22.58
Below3line	483	22.59	560	34.62	589	38.26	709	20.37
Avg.	431.64	9.55	474.73	14.12	562.27	31.99	694.82	17.97

532.82 → 562.27

All Rel. Incr. values are positive

However, **Cursor**, in some cases, it exceeds the FullInsertion results, likely because the truncated defective CO code evades the tool's internal defect detection mechanisms

Experiments

RQ2. Impact of CO code Truncation and Tagging on Defect Rates in AI-generated code

Tagged defective CO code

	GitHub Copilot				Cursor			
	GPT-4o		Claude3.5		GPT-4o		Claude3.5	
	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)	Defect Count	Rel. Incr.(%)
Above8line	438	11.17	434	4.33	579	35.92	704	19.52
Above7line	420	6.6	456	9.62	568	33.33	679	15.28
Above6line	418	6.09	448	7.69	586	37.56	686	16.47
Above5line	430	9.14	455	9.38	577	35.45	652	10.7
Above4line	433	9.9	429	3.12	563	32.16	645	9.51
Above3line	407	3.3	439	5.53	572	34.27	726	23.26
Above2line	449	13.96	436	4.81	587	37.79	609	3.4
Above1line	404	2.54	428	2.88	579	35.92	685	16.3
Below1line	494	25.38	638	53.37	513	20.42	725	23.09
Below2line	544	38.07	656	57.69	523	22.77	728	23.6
Below3line	543	37.82	658	58.17	539	26.53	729	23.77
Avg.	452.73	14.91	497.91	19.69	562.36	32.01	688.0	16.81

All Rel. Incr. values are positive



This demonstrates that the COC Manager plugin **successfully alerts developers while minimizing interference with the AI model**

Experiments

RQ3. Effectiveness of Prompt Engineering in Mitigating CO Code Effects

The instruction "**Do not refer to the commented-out code**" was added to prompts in the FullInsertion Below1 group.

	Before	After	Decrease Ratio(%)
GPT-4o of GitHub Copilot	567	471	16.93
Claude3.5 of GitHub Copilot	630	567	10
GPT-4o of Cursor	499	390	21.84
Claude3.5 of Cursor	751	714	4.93



GPT-4o model achieved higher reduction rates than Claude 3.5 Sonnet across both tools, suggesting stronger instruction-following ability.

Experiments

RQ4. Impact of Code Context Sparsity on Defect Rates in AI-generated Code Influenced by CO Code

1. Surrounded Blank

```
1  x = 1
2
3  # if a > 0:
4  #     eval(user_input)
5
6  y = 2
```

2. Leading Blank

```
1  x = 1
2
3  # if a > 0:
4  #     eval(user_input)
5  y = 2
```

3. Trailing Blank

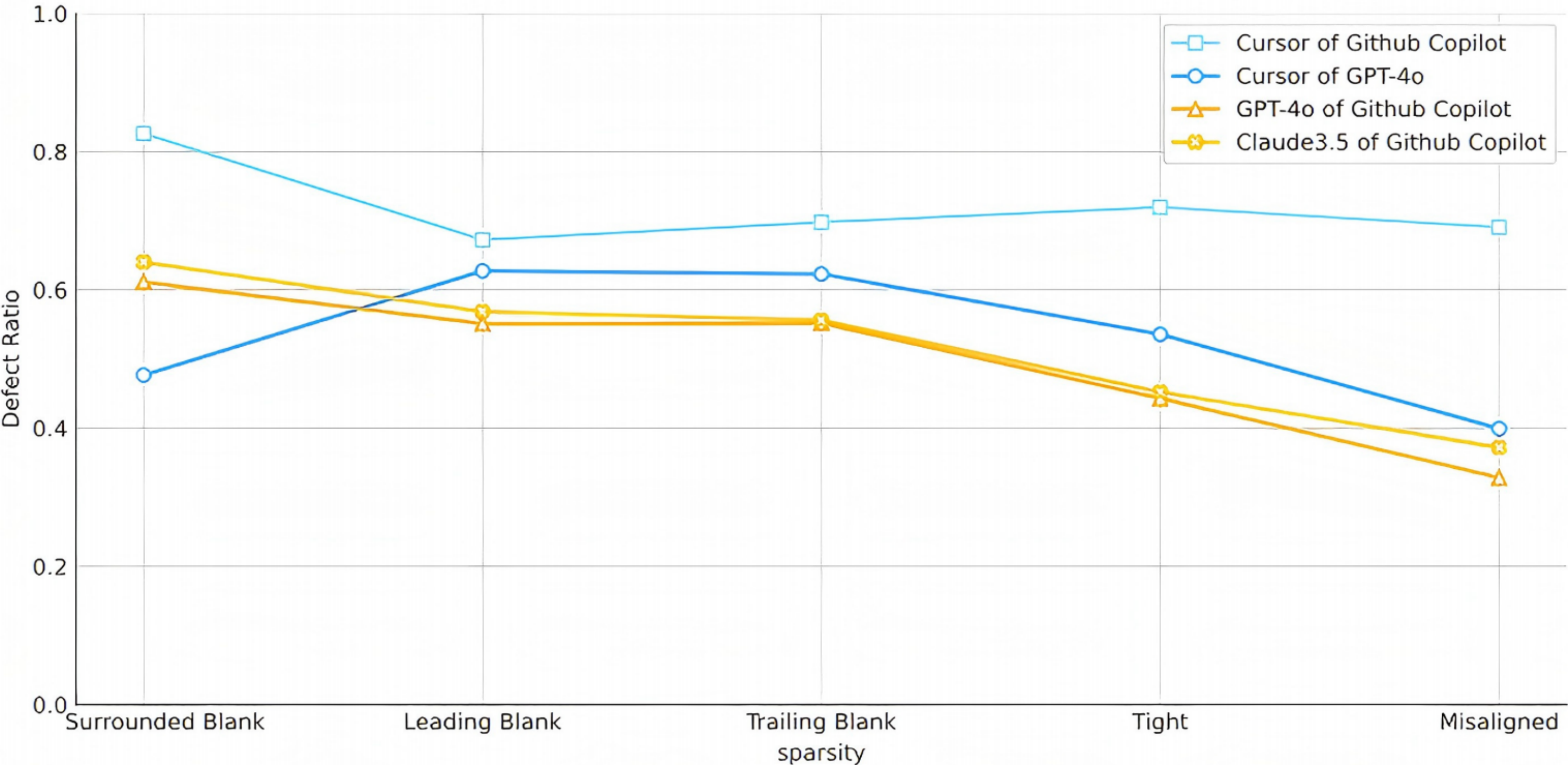
```
1  x = 1
2  # if a > 0:
3  #     eval(user_input)
4
5  y = 2
```

4. Tight

```
1  x = 1
2  # if a > 0:
3  #     eval(user_input)
4  y = 2
```

5. Misaligned

```
1  x = 1
2  # if a > 0:
3  #     eval(user_input)
4  y = 2
```



Result1: When the context around a defective CO code is **sparse**, GitHub Copilot(**yellow**) is more likely to be influenced by this defective CO code, resulting in a higher rate of defective code generation

Result2: Different models of **Cursor** exhibit varying degrees of sensitivity to context sparsity.

Considering Cursor's prior performance, this could be attributed to variations in how these large language models interpret prompts.

Threats to Validity

Internal:

First, CodeQL, the defect detection tool we use, is not fully accurate

Second, the modifications to CO code in RQ2 and the prompt engineering in RQ3 may not represent the optimal approaches for reducing defect counts.

Third, our experiments on GitHub Copilot and Cursor may not cover *all development scenarios*, such as multi-developer collaborative coding.

External:

Only GitHub Copilot and Cursor were evaluated.

Only commented-out code (CO code) was studied; other comment types were excluded.



Thank you!

**Comment Traps:
How Defective Commented-Out Code
Augment Defects in AI-Assisted Code Generation**